Assignment-14.5

Name:-U.Vignesh

Ht no:- 2303A51964

Batch-24

Task 1 – Image Gallery Website

Create an image gallery web page.

Requirements:

• Display images in a grid layout.

• Use AI to:

o Optimize layout using CSS Grid.

o Add hover zoom or overlay effects.

o Ensure mobile responsiveness.

```
<!--Create an image gallery web page and Display images in a grid layout,Optimize layout using CSS Grid,Add hover zoom or overlay effects.Ensure mobile responsiveness.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Image Gallery</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            padding: 0;
            background-color: #f0f0f0;
        }
        .gallery {
            display: grid;
            grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
            gap: 15px;
            padding: 20px;
        }
        .gallery-item {
            position: relative;
            overflow: hidden;
        }
        .gallery-item img {
            width: 100%;
            height: auto;
            transition: transform 0.3s ease;
        }
        .gallery-item:hover img {
            transform: scale(1.1);
        }
        @media (max-width: 600px) {
            .gallery {
                grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
            }
        }
```

```
                grid-template-columns: repeat(auto-fit, minmax(150px, 1fr));
            }
        }
    </style>
</head>
<body>
    <h1 style="text-align: center; padding: 20px;">Image Gallery</h1>
    <div class="gallery">
        <div class="gallery-item">
            <img src="https://via.placeholder.com/400x300" alt="Image 1">
        </div>
        <div class="gallery-item">
            <img src="https://via.placeholder.com/400x300" alt="Image 2">
        </div>
        <div class="gallery-item">
            <img src="https://via.placeholder.com/400x300" alt="Image 3">
        </div>
        <div class="gallery-item">
            <img src="https://via.placeholder.com/400x300" alt="Image 4">
        </div>
        <div class="gallery-item">
            <img src="https://via.placeholder.com/400x300" alt="Image 5">
        </div>
        <div class="gallery-item">
            <img src="https://via.placeholder.com/400x300" alt="Image 6">
        </div>
    </div>
</body>
</html>
```

OUTPUT:

Task 2 – Profile Card Generator

Create a profile card web page.

Requirements:

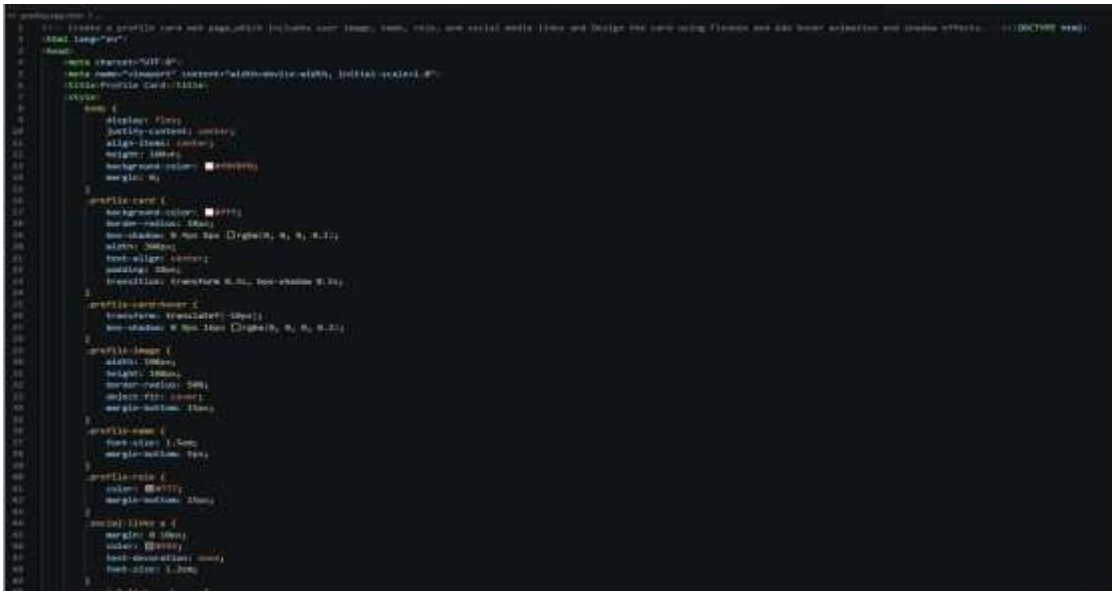
• Display user image, name, role, and social media links.

• Use AI to:

o Suggest modern UI colors and fonts.

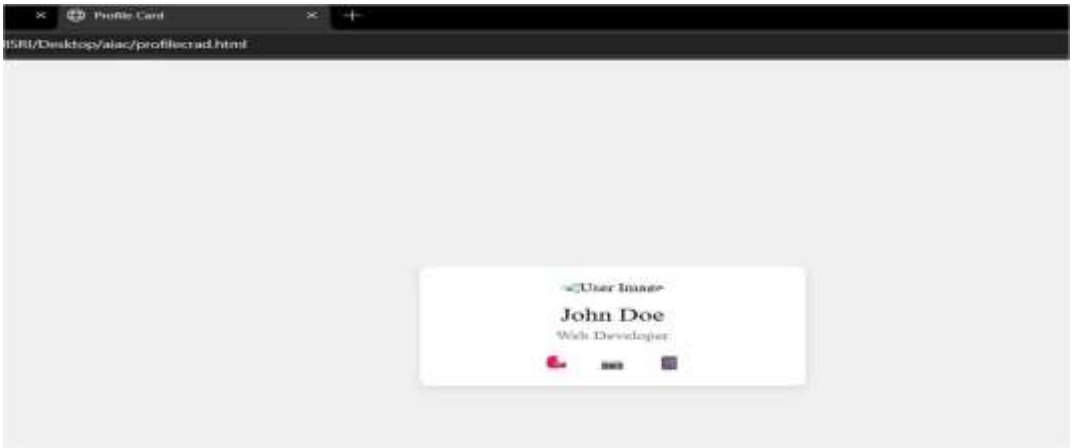
o Design the card using Flexbox.

o Add hover animation and shadow effects



```
        margin-bottom: 15px;
    }
    .profile-name {
        font-size: 1.5em;
        margin-bottom: 5px;
    }
    .profile-role {
        color: #777;
        margin-bottom: 15px;
    }
    .social-links a {
        margin: 0 10px;
        color: #555;
        text-decoration: none;
        font-size: 1.2em;
    }
    .social-links a:hover {
        color: #000;
    }
</style>
</head><body>
<div class="profile-card">
    <img src="https://via.placeholder.com/100" alt="User Image" class="profile-image">
    <div class="profile-name">John Doe</div>
    <div class="profile-role">Web Developer</div>
    <div class="social-links">
        <a href="#">&#x1F426;</a>
        <a href="#">&#x1F4F7;</a>
        <a href="#">&#x1F4F1;</a>
    </div>
</div>
</body></html>
```

OUTPUT:



Task 3 -Develop a dynamic shopping cart system.

Requirements:

• Add, remove, and update product quantities.

• Display real-time total price.

• Use AI to:

o Manage cart state using JavaScript objects.

o Persist cart data in localStorage.

```
shopping-cart-system.html > ⓗtml > ⓑody
<!-Develop a dynamic shopping cart system with Add, remove, and update product quantitie,Display real-time total price and also Manage cart state using JavaScript objects and localStor
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic Shopping Cart</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .product {
            display: flex;
            justify-content: space-between;
            margin-bottom: 10px;
        }
        .cart {
            margin-top: 20px;
        }
        .cart-item {
            display: flex;
            justify-content: space-between;
            margin-bottom: 10px;
        }
        .total {
```

```
        .total {
        }
    </style>
</head><body>
    <h1>Shopping Cart</h1>
    <div id="products">
        <div class="product">
            <span>Product 1 - $10</span>
            <button onclick="addToCart('Product 1', 10)">Add to Cart</button>
        </div>
        <div class="product">
            <span>Product 2 - $20</span>
            <button onclick="addToCart('Product 2', 20)">Add to Cart</button>
        </div>
        <div class="product">
            <span>Product 3 - $30</span>
            <button onclick="addToCart('Product 3', 30)">Add to Cart</button>
        </div>
    </div>
    <div class="cart">
        <h2>Your Cart</h2>
        <div id="cart-items"></div>
        <div class="total">Total: $<span id="total-price">0</span></div>
    </div>
    <script>
        let cart = JSON.parse(localStorage.getItem('cart')) || {};

        function addToCart(productName, price) {
            if (cart[productName]) {
                cart[productName].quantity += 1;
            } else {
                cart[productName] = { price: price, quantity: 1 };
```

```
        <div class="product">
        </div>
    </div>
    <div class="cart">
        <h2>Your Cart</h2>
        <div id="cart-items"></div>
        <div class="total">Total: $<span id="total-price">0</span></div>
    </div>
    <script>
        let cart = JSON.parse(localStorage.getItem('cart')) || {};

        function addToCart(productName, price) {
            if (cart[productName]) {
                cart[productName].quantity += 1;
            } else {
                cart[productName] = { price: price, quantity: 1 };
            }
            updateCart();
        }

        function removeFromCart(productName) {
            delete cart[productName];
            updateCart();
        }

        function updateQuantity(productName, quantity) {
            if (quantity <= 0) {
                removeFromCart(productName);
            } else {
                cart[productName].quantity = quantity;
                updateCart();
            }
```

```
        }

        function updateCart() {
            const cartItemsContainer = document.getElementById('cart-items');
            cartItemsContainer.innerHTML = '';
            let totalPrice = 0;

            for (let product in cart) {
                const item = cart[product];
                totalPrice += item.price * item.quantity;

                const cartItem = document.createElement('div');
                cartItem.classList.add('cart-item');
                cartItem.innerHTML = `
                    <span>${product} - $${item.price} x ${item.quantity}</span>
                    <input type="number" value="${item.quantity}" min="1" onchange="updateQuantity('${product}', this.value)">
                    <button onclick="removeFromCart('${product}')">Remove</button>
                `;
                cartItemsContainer.appendChild(cartItem);
            }

            document.getElementById('total-price').textContent = totalPrice.toFixed(2);
            localStorage.setItem('cart', JSON.stringify(cart));
        }

        // Initialize the cart on page load
        updateCart();
    </script></body>
</html>
```

OUTPUT:

Task 4 – Role-Based Access Control Application

Create a role-based web application.

Requirements:

• Admin, Editor, User roles.

• Conditional UI rendering.

• Use AI to:

o Design access control logic.

o Maintain secure state transitions.

o Create scalable UI components.

```html
<!--Create a role-based web application with Admin, Editor, User roles and Conditional UI rendering and alsoDesign access control logic,Maintain secure state transitions,Create scalable
<DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Role-Based Web Application</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .navbar {
            background-color: #333;
            overflow: hidden;
        }
        .navbar a {
            float: left;
            display: block;
            color: white;
            text-align: center;
            padding: 14px 16px;
            text-decoration: none;
        }
        .navbar a:hover {
            background-color: #ddd;
            color: black;
        }
        .content {
            margin-top: 20px;
        }
    </style>
</head><body>
```

```html
<html lang="en">
<head>
    <style>
        }
    </style>
</head><body>
    <div class="navbar">
        <a href="#home">Home</a>
        <a href="#admin" id="admin-link" style="display: none;">Admin Panel</a>
        <a href="#editor" id="editor-link" style="display: none;">Editor Dashboard</a>
        <a href="#user" id="user-link" style="display: none;">User Profile</a>
    </div>
    <div class="content" id="content">
        <h1>Welcome to the Role-Based Web Application</h1>
        <p>Please log in to see your dashboard.</p>
    </div>
    <script>
        // Simulated user role (can be 'admin', 'editor', or 'user')
        const userRole = 'editor'; // Change this value to test different roles

        // Function to render UI based on user role
        function renderUI(role) {
            document.getElementById('admin-link').style.display = role === 'admin' ? 'block' : 'none';
            document.getElementById('editor-link').style.display = role === 'editor' ? 'block' : 'none';
            document.getElementById('user-link').style.display = role === 'user' ? 'block' : 'none';

            const content = document.getElementById('content');
            if (role === 'admin') {
                content.innerHTML = '<h1>Admin Panel</h1><p>Manage users, settings, and more.</p>';
            } else if (role === 'editor') {
                content.innerHTML = '<h1>Editor Dashboard</h1><p>Create and manage content.</p>';
            } else if (role === 'user') {
                content.innerHTML = '<h1>User Profile</h1><p>View and edit your profile.</p>';
            } else {
                content.innerHTML = '<h1>Welcome to the Role-Based Web Application</h1><p>Please log in to see your dashboard.</p>';
            }
```

Task 5 – Multi-Language Website

Design a multilingual website.

Requirements:

• Switch languages dynamically.

• Persist language preference.

• Use AI to:

o Structure translation files.

o Handle RTL/LTR layouts.

o Improve accessibility.

```
<!--Design a multilingual website that Switch languages dynamically and Persist language preference,Handle RTL/LTR layouts and Ensure accessibility for all users.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Multilingual Website</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .language-switcher {
            margin-bottom: 20px;
        }
        .rtl {
            direction: rtl;
        }
        .ltr {
            direction: ltr;
        }
    </style>
</head>
<body>
    <div class="language-switcher">
        <label for="language">Choose language:</label>
        <select id="language" onchange="switchLanguage()">
            <option value="en">English</option>
            <option value="ar">Arabic</option>
            <option value="fr">French</option>
        </select>
    </div>

    <h1 id="greeting">Hello, welcome to our website!</h1>
    <p id="description">This is a multilingual website that supports dynamic language switching and persists user preferences.</p>
```

```
<script>
    function switchLanguage() {
        const language = document.getElementById('language').value;
        localStorage.setItem('preferredLanguage', language);
        updateContent(language);
    }

    function updateContent(language) {
        const greeting = document.getElementById('greeting');
        const description = document.getElementById('description');
        const body = document.body;

        if (language === 'en') {
            greeting.textContent = 'Hello, welcome to our website!';
            description.textContent = 'This is a multilingual website that supports dynamic language switching and persists user preferences.';
            body.className = 'ltr';
        } else if (language === 'ar') {
            greeting.textContent = 'مرحبًا، مرحبًا بكم في موقعنا!';
            description.textContent = 'هذا موقع متعدد اللغات يدعم التبديل الديناميكي للغة ويحفظ تفضيلات المستخدم.';
            body.className = 'rtl';
        } else if (language === 'fr') {
            greeting.textContent = 'Bonjour, bienvenue sur notre site web!';
            description.textContent = 'Ceci est un site web multilingue qui prend en charge le changement de langue dynamique et persiste les préférences des utilisateurs.';
            body.className = 'ltr';
        }
    }

    // Load preferred language on page load
    window.onload = function() {
        const preferredLanguage = localStorage.getItem('preferredLanguage') || 'en';
        document.getElementById('language').value = preferredLanguage;
        updateContent(preferredLanguage);
    };
</script></body>
</html>
```

OUTPUT:

Task 6 – Online Examination System

Create an online exam interface.

Requirements:

• Timer, questions, and submission.

• Prevent multiple submissions.

• Use AI to:

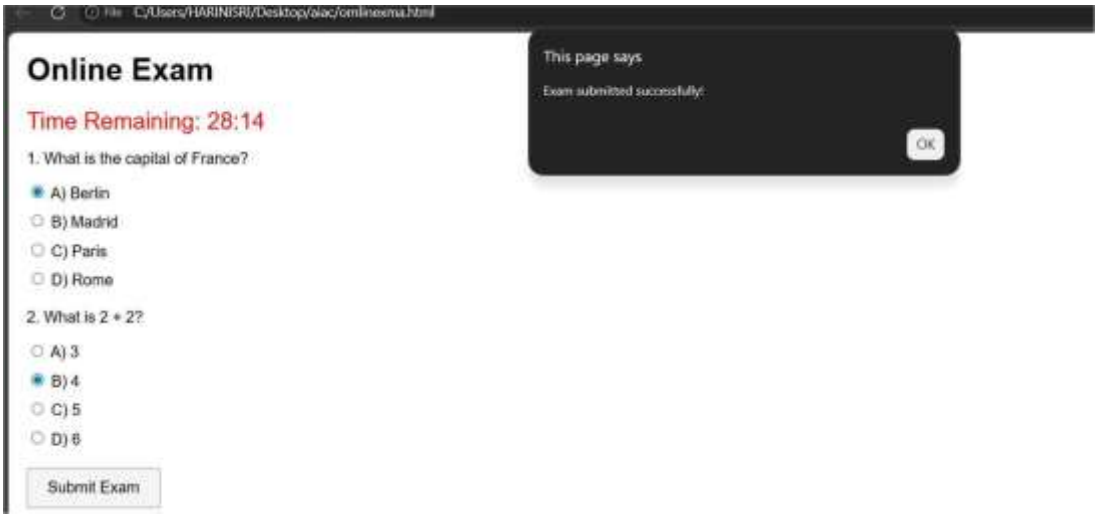
o Manage exam state.

o Improve exam UX.

o Ensure accessibility.

```html
<!--Create an online exam interface with timer, questions, and submission and Prevent multiple submissions,Manage exam state,Improve exam UX and ensure accessibility.-->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Online Exam Interface</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 20px;
        }
        .timer {
            font-size: 20px;
            color: red;
        }
        .question {
            margin-bottom: 20px;
        }
        .options {
            list-style-type: none;
            padding: 0;
        }
        .options li {
            margin-bottom: 10px;
        }
        .submit-btn {
            padding: 10px 20px;
            font-size: 16px;
        }
    </style>
</head><body>
    <h1>Online Exam</h1>
    <div class="timer" id="timer">Time Remaining: 30:00</div>
```

```html
</head><body>
    <h1>Online Exam</h1>
    <div class="timer" id="timer">Time Remaining: 30:00</div>
    <form id="examForm">
        <div class="question">
            <p>1. What is the capital of France?</p>
            <ul class="options">
                <li><input type="radio" name="q1" value="a"> A) Berlin</li>
                <li><input type="radio" name="q1" value="b"> B) Madrid</li>
                <li><input type="radio" name="q1" value="c"> C) Paris</li>
                <li><input type="radio" name="q1" value="d"> D) Rome</li>
            </ul>
        </div>
        <div class="question">
            <p>2. What is 2 + 2?</p>
            <ul class="options">
                <li><input type="radio" name="q2" value="a"> A) 3</li>
                <li><input type="radio" name="q2" value="b"> B) 4</li>
                <li><input type="radio" name="q2" value="c"> C) 5</li>
                <li><input type="radio" name="q2" value="d"> D) 6</li>
            </ul>
        </div>
        <button type="submit" class="submit-btn">Submit Exam</button>
    </form>
    <script>
        // Timer functionality
        let timeRemaining = 30 * 60; // 30 minutes in seconds
        const timerElement = document.getElementById('timer');
        const examForm = document.getElementById('examForm');
        let examSubmitted = false;

        function updateTimer() {
            if (timeRemaining <= 0) {
                submitExam();
                return;
```

```
<script>
        const examForm = document.getElementById('examForm');
        let examSubmitted = false;

        function updateTimer() {
            if (timeRemaining <= 0) {
                submitExam();
                return;
            }
            const minutes = Math.floor(timeRemaining / 60);
            const seconds = timeRemaining % 60;
            timerElement.textContent = `Time Remaining: ${minutes}:${seconds < 10 ? '0' : ''}${seconds}`;
            timeRemaining--;
        }

        setInterval(updateTimer, 1000);

        // Prevent multiple submissions
        examForm.addEventListener('submit', function(event) {
            event.preventDefault();
            if (examSubmitted) {
                alert('You have already submitted the exam.');
                return;
            }
            submitExam();
        });

        function submitExam() {
            examSubmitted = true;
            alert('Exam submitted successfully!');
            // Here you can add code to send the answers to the server
        }
    </script>
</body></html>
```

OUTPUT: